

Vibe Coding Log - AttritionIQ

Built 7-9 October 2026, every change merged into `main` through a pull request. Full record with every entry: `docs/VIBE_CODING_LOG_FULL.md`.

Platforms. **Claude** (a `claude.ai` Project holding the brief and the plan, plus agent sessions working in the repo on the laptop): design, phase plans, code, tests, docs and live-site checks. **GitHub**: one branch per change, pull requests into `main`. **Google AI Studio**: Gemini key for the HR copilot brief. **Streamlit Community Cloud**: hosting. **Workflow.** For each phase: (1) give the AI the rubric and the current results; (2) the AI drafts a plan whose decision rules and pass/fail checks are written down *before* any run (`docs/PHASE3_PLAN.md`, `PHASE5_PLAN.md`, `PHASE6_PLAN.md`); (3) we review the plan; (4) the AI implements and runs it; (5) we verify the result ourselves, then merge through a pull request.

Prompt strategies that worked

- Rubric first, then design, then code. Asking for the design first avoided rework.
- "Don't invent data": every assumption is labelled with a source or as a team choice; the HR handbook cites EU/UK law and GitLab's public handbook.
- Fix the selection rule before seeing results (e.g. "simplest model within 1 std wins"), so the AI cannot tune the story to the numbers.
- Ask for every number in a write-up to be checked against `reports/metrics.json` by script, not by reading.
- Ask for an honest baseline or ceiling for every claim (naive forecast, item-mean, noise-free oracle).

Phase	What the AI did	What we changed or rejected	How we verified
0-1 Setup, EDA	Repo skeleton, data checks, feature ablation	Asked for a paired, two-model ablation; all 4 engineered features dropped	pytest; ablation numbers reproduced
2 Risk model	4-model comparison, nested CV, calibration, threshold, SHAP, fairness	Rules in code before the full run; rejected dropping income to "fix" SHAP signs (costs PR-AUC, $p = 0.045$)	Threshold checked against the break-even formula; numbers checked by script
3 Other tracks	Segments, recommender, forecast, RAG with a 33-question eval	Kept the naive forecast because ARIMA lost in rolling CV; rejected SHAP clusters	Oracle ceiling for CF; JOLTS checked against FRED
4 Front-end	4 task-based pages, shared assumptions panel	Restyled Streamlit instead of a separate JS front-end, so everything stays in Python	Headless run of every page and interaction
5 Freeze	Fresh-clone reproducibility, app tests	Froze the README-only run so graders can reproduce it	Fresh clone: 25 tests passed; planted bug caught
6 Deploy	Host choice, model bundle, live checks, LLM timeout	Dropped Vercel and the custom domain; accepted the template brief when Gemini is overloaded	Live numbers match the frozen model; 26 tests passed

What the AI got wrong, and how we caught it

- **Seed collision**: the recommender "beat" a perfect oracle, which is impossible. Funk SVD and the simulator both used seed 42. Fixed with separate random streams and a regression test.
- **ARIMA order**: AIC was compared across different differencing orders, which is invalid. Now d comes from ADF + KPSS first.
- **Age bands**: `pd.cut` put 50-year-olds in "40-49". Caught when the 50+ count did not match `Age >= 50`.
- **No LLM timeout**: the live brief hung for 4+ minutes. Neither SDK sets a timeout by default; we added one with a fallback and a test.

Decided by us, not the AI: persona names (team vote), excluding sensitive attributes, keeping the default cost assumptions, Logistic Regression by the pre-set rule, TF-IDF on the host, the free hosting tier.